

Estructuras de Datos Lineales



Tarea 2:

Uso de `std::vector` en C++

Profesor:

Dr. Erik Reyes Reyes

Alumno:

Alemán Baza Dante

Fecha:

Jueves 13 de mayo de 2026

Objetivo de la Tarea

Familiarizar al estudiante con el manejo dinámico de memoria, la gestión de la capacidad y el uso de iteradores mediante la librería estándar `<vector>` de C++.

Ejercicios Propuestos

Ejercicio 1:

Escribe un programa que simule el registro de calificaciones de un grupo.

1. Crea un `vector<float>` vacío.
2. Solicita al usuario que ingrese 5 calificaciones y agrégalas al vector usando `push_back()`.
3. Recorre el vector y calcula el promedio del grupo.
4. Encuentra la calificación más baja y elimínala usando el método `erase()`.
5. Imprime el tamaño actual del vector (`size()`) y las calificaciones restantes.

```
#include <iostream>
#include <vector>

using namespace std;

void ingresaCalificaciones(vector<float>& cals, int n);
float calculaPromedio(vector<float>& cals);
int indiceCalificacionMenor(vector<float>& cals);
void printVector(const vector<float>& cals);

/**
 * Programa que simula el registro de calificaciones de un grupo.
 */
int main() {
    // 1. Crea un vector<float> vacio.
    vector<float> calificaciones;

    // 2. Solicita al usuario que ingrese 5 calificaciones y agregalas
    // al vector usando push back().
    ingresaCalificaciones(calificaciones, 5);
    cout << "Se ingresaron correctamente las calificaciones: " <<
endl;
    printVector(calificaciones);

    // 3. Recorre el vector y calcula el promedio del grupo.
    float promedio = calculaPromedio(calificaciones);
```

```

        // 4. Encuentra la calificacion mas baja y eliminala usando el
        metodo erase().

        int indice_cal_menor = indiceCalificacionMenor(calificaciones);
        calificaciones.erase(calificaciones.begin() + indice_cal_menor);

        // 5. Imprime el tamaño actual del vector (size()) y las
        calificaciones restantes.

        cout << "El vector calificaciones tiene un tamaño = " <<
        calificaciones.size() << " despues de usar la funcion erase()" <<
        endl;
        printVector(calificaciones);
    }

void ingresaCalificaciones(vector<float>& cals, int n) {
    float aux = 0.0f;
    for (int i = 0; i < n; i++) {
        cout << "Ingresa la calificacion " << (i + 1) << ": ";
        cin >> aux;
        cals.push_back(aux);
    }
}

float calculaPromedio(vector<float>& cals) {
    float suma = 0.0f;
    int n = cals.size();
    for (int i = 0; i < n; i++) {
        suma += cals[i];
    }
    float promedio = suma / n;
    return promedio;
}

int indiceCalificacionMenor(vector<float>& cals) {
    int indice_cal_menor = 0;
    for (int i = 0; i < cals.size(); i++) {
        if (cals[i] < cals[indice_cal_menor]) {
            indice_cal_menor = i;
        }
    }
    return indice_cal_menor;
}

void printVector(const vector<float>& cals) {
    cout << "[";
    for (int i = 0; i < cals.size() - 1; i++) {
        cout << cals[i] << ", ";
    }
}

```

```

        cout << cals.back() << "]" << endl;
    }
}

```

Compilación y ejecución:

```

dalemanb@generic:/mnt/c/Users/dalem/Desktop/Cpp/C_plus_plus/Homework/Tarea2$ g++ Ejercicio1.cpp
dalemanb@generic:/mnt/c/Users/dalem/Desktop/Cpp/C_plus_plus/Homework/Tarea2$ ./a.out
Ingresa la calificacion 1: 9.3
Ingresa la calificacion 2: 6.3
Ingresa la calificacion 3: 8.4
Ingresa la calificacion 4: 7.9
Ingresa la calificacion 5: 8.8
Se ingresaron correctamente las calificaciones:
[9.3, 6.3, 8.4, 7.9, 8.8]
El vector calificaciones tiene un tamaño = 4 despues de usar la funcion erase()
[9.3, 8.4, 7.9, 8.8]

```

Ejercicio 2:

Este ejercicio busca demostrar cómo crece dinámicamente un vector en memoria.

1. Crea un `vector<int>`.
2. Imprime su tamaño (`size`) y capacidad (`capacity`) iniciales.
3. Utiliza un ciclo para agregar 100 elementos continuos usando `push_back()`. Dentro del bucle, imprime en consola cada vez que la capacidad del vector cambie.
4. Una vez finalizado el bucle, elimina con `erase()` los elementos de indice impar.
5. Llama a la función `shrink_to_fit()` para liberar la memoria RAM no utilizada y comprueba imprimiendo nuevamente la capacidad final.

```

#include <iostream>
#include <vector>

using namespace std;

void llenaVector(vector<int>& vect, int n);
void eliminaImpares(vector<int>& vect);
void printVector(const vector<int>& vect);

/**
 * Programa que busca demostrar como crece dinamicamente un vector en
 memoria.
 */
int main() {
    // 1. Crea un vector<int>.
    vector<int> mi_vector;
    cout << "Vector creado." << endl;

    // 2. Imprime su tamaño (size) y capacidad (capacity) iniciales.
    cout << "El vector tiene un tamaño inicial = " <<
mi_vector.size()

```

```

        << " y una capacidad inicial = " << mi_vector.capacity() <<
endl;

    // 3. Utiliza un ciclo para agregar 100 elementos continuos usando
push back(). Dentro del
    // bucle, imprime en consola cada vez que la capacidad del vector
cambie.
    llenaVector(mi_vector, 100);

    // 4. Una vez finalizado el bucle, elimina con erase() los
elementos de indice impar.
    eliminaImpares(mi_vector);

    // 5. Llama a la funcion shrink to fit() para liberar la memoria
RAM no utilizada y com-
    // prueba imprimiendo nuevamente la capacidad final.
    mi_vector.shrink_to_fit();
    cout << "El vector tiene un tamaño final = " << mi_vector.size()
        << " y una capacidad final = " << mi_vector.capacity() <<
endl;
}

void llenaVector(vector<int>& vect, int n) {
    for (int i = 0; i < n; i++) {
        int valor = i + 1;
        vect.push_back(valor);
        cout << "El vector tiene un tamaño = " << vect.size()
            << " y una capacidad = " << vect.capacity() << endl;
    }
}

void eliminaImpares(vector<int>& vect) {
    for (int i = 0; i < vect.size(); i++) {
        if (vect[i] % 2 == 1) {
            vect.erase(vect.begin() + i);
        }
    }
}

void printVector(const vector<int>& vect) {
    cout << "[";
    for (int i = 0; i < vect.size() - 1; i++) {
        cout << vect[i] << ", ";
    }
    cout << vect.back() << "]" << endl;
}

```


Compilación y ejecución:

```
dalemanb@generic:/mnt/c/Users/dalem/Desktop/Cpp/C_plus_plus/Homework/Tarea2$ g++ Ejercicio2.cpp
dalemanb@generic:/mnt/c/Users/dalem/Desktop/Cpp/C_plus_plus/Homework/Tarea2$ ./a.out
Vector creado.
El vector tiene un tamaño inicial = 0 y una capacidad inicial = 0
El vector tiene un tamaño = 1 y una capacidad = 1
El vector tiene un tamaño = 2 y una capacidad = 2
El vector tiene un tamaño = 3 y una capacidad = 4
El vector tiene un tamaño = 4 y una capacidad = 4
El vector tiene un tamaño = 5 y una capacidad = 8
El vector tiene un tamaño = 6 y una capacidad = 8
El vector tiene un tamaño = 7 y una capacidad = 8
El vector tiene un tamaño = 8 y una capacidad = 8
El vector tiene un tamaño = 9 y una capacidad = 16
El vector tiene un tamaño = 10 y una capacidad = 16
El vector tiene un tamaño = 11 y una capacidad = 16
El vector tiene un tamaño = 12 y una capacidad = 16
El vector tiene un tamaño = 13 y una capacidad = 16
El vector tiene un tamaño = 14 y una capacidad = 16
El vector tiene un tamaño = 15 y una capacidad = 16
El vector tiene un tamaño = 16 y una capacidad = 16
El vector tiene un tamaño = 17 y una capacidad = 32
El vector tiene un tamaño = 18 y una capacidad = 32
El vector tiene un tamaño = 19 y una capacidad = 32
El vector tiene un tamaño = 20 y una capacidad = 32
El vector tiene un tamaño = 21 y una capacidad = 32
El vector tiene un tamaño = 22 y una capacidad = 32
El vector tiene un tamaño = 23 y una capacidad = 32
El vector tiene un tamaño = 24 y una capacidad = 32
El vector tiene un tamaño = 25 y una capacidad = 32
El vector tiene un tamaño = 26 y una capacidad = 32
El vector tiene un tamaño = 27 y una capacidad = 32
El vector tiene un tamaño = 28 y una capacidad = 32
El vector tiene un tamaño = 29 y una capacidad = 32
El vector tiene un tamaño = 30 y una capacidad = 32
El vector tiene un tamaño = 31 y una capacidad = 32
El vector tiene un tamaño = 32 y una capacidad = 32
El vector tiene un tamaño = 33 y una capacidad = 64
El vector tiene un tamaño = 34 y una capacidad = 64
El vector tiene un tamaño = 35 y una capacidad = 64
El vector tiene un tamaño = 36 y una capacidad = 64
El vector tiene un tamaño = 37 y una capacidad = 64
El vector tiene un tamaño = 38 y una capacidad = 64
El vector tiene un tamaño = 39 y una capacidad = 64
El vector tiene un tamaño = 40 y una capacidad = 64
El vector tiene un tamaño = 41 y una capacidad = 64
El vector tiene un tamaño = 42 y una capacidad = 64
El vector tiene un tamaño = 43 y una capacidad = 64
El vector tiene un tamaño = 44 y una capacidad = 64
El vector tiene un tamaño = 45 y una capacidad = 64
El vector tiene un tamaño = 46 y una capacidad = 64
El vector tiene un tamaño = 47 y una capacidad = 64
El vector tiene un tamaño = 48 y una capacidad = 64
El vector tiene un tamaño = 49 y una capacidad = 64
El vector tiene un tamaño = 50 y una capacidad = 64
El vector tiene un tamaño = 51 y una capacidad = 64
El vector tiene un tamaño = 52 y una capacidad = 64
El vector tiene un tamaño = 53 y una capacidad = 64
El vector tiene un tamaño = 54 y una capacidad = 64
El vector tiene un tamaño = 55 y una capacidad = 64
El vector tiene un tamaño = 56 y una capacidad = 64
```

[illegible]

Ejercicio 3:

Implementa un directorio que mantenga a los empleados ordenados automáticamente por su número de identificación (ID) al momento de ser ingresados.

1. Define un `struct Empleado` que contenga: `string nombre`, `int id`, y `float salario`.
2. En la función `main`, crea un `vector<Empleado>` vacío.
3. Crea manualmente cuatro instancias de la estructura `Empleado` asignándoles IDs desordenados (por ejemplo: 105, 102, 109, 101).
4. Implementa la lógica de inserción ordenada: para agregar cada empleado, recorre el vector para encontrar la posición exacta donde debe ir el nuevo registro, garantizando que el vector se mantenga ordenado de menor a mayor respecto al `id`.
5. Utiliza el método `insert()` pasando el iterador calculado y la instancia del empleado previamente construida.
6. Imprime el directorio completo recorriéndolo exclusivamente con iteradores para demostrar que los empleados quedaron correctamente ordenados por su ID.

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

//1. Define un struct Empleado que contenga: string nombre, int id, y
float salario
struct Empleado {
    string nombre;
    int id;
    float salario;

    Empleado(string n, int i, float s) : nombre(n), id(i), salario(s)
{}
};

void insertaOrdenado(vector<Empleado>& original, vector<Empleado>&
destino);
void printVector(vector<Empleado>& vect);

/**
 * Programa que implementa un directorio que mantenga a los empleados
ordenados automaticamente por su
 * numero de identificacion (ID) al momento de ser ingresados
```



```

*/
int main() {
    //2. En la funcion main, crea un vector<Empleado> vacio
    vector<Empleado> empleados;

    //3. Crea manualmente cuatro instancias de la estructura Empleado
    asignandoles IDs desorde-
    //nados (por ejemplo: 105, 102, 109, 101)
    Empleado e1 = {"Ana", 105, 3500.50f};
    Empleado e2 = {"Luis", 102, 4200.00f};
    Empleado e3 = {"Carlos", 109, 3900.75f};
    Empleado e4 = {"Maria", 101, 4500.25f};

    empleados.push_back(e1);
    empleados.push_back(e2);
    empleados.push_back(e3);
    empleados.push_back(e4);

    cout << "Vector original:\n";
    printVector(empleados);

    // 4. Implementa la logica de insercion ordenada: para agregar
    cada empleado, recorre el vector
    // para encontrar la posicion exacta donde debe ir el nuevo
    registro, garantizando que el
    // vector se mantenga ordenado de menor a mayor respecto al id.
    // 5. Utiliza el metodo insert() pasando el iterador calculado y
    la instancia del empleado
    // previamente construida.
    vector<Empleado> directorio;
    insertaOrdenado(empleados, directorio);

    // 6. Imprime el directorio completo recorriendolo exclusivamente
    con iteradores para demostrar
    // que los empleados quedaron correctamente ordenados por su ID.
    cout << "\nVector ordenado:\n";
    printVector(directorio);

    return 0;
}

void insertaOrdenado(vector<Empleado>& original, vector<Empleado>&
destino) {
    for (size_t i = 0; i < original.size(); i++) {
        Empleado actual = original[i];
        vector<Empleado>::iterator it = destino.begin();
        while (it != destino.end() && it->id < actual.id) {

```

```

        it++;
    }
    destino.insert(it, actual);
}
}

void printVector(vector<Empleado>& vect) {
    cout << "--- Directorio de Empleados ---" << endl;

    for (vector<Empleado>::iterator it = vect.begin(); it !=
vect.end(); it++) {
        cout << "ID      : " << it->id << endl;
        cout << "Nombre  : " << it->nombre << endl;
        cout << "Salario : $" << it->salario << endl;
        cout << endl;
    }
}
}

```

Compilación y ejecución:

```

dalemanb@generic:/mnt/c/Users/dalem/Desktop/Cpp/C_plus_plus/Homework/Tarea2$ g++ Ejercicio3.cpp
dalemanb@generic:/mnt/c/Users/dalem/Desktop/Cpp/C_plus_plus/Homework/Tarea2$ ./a.out
Vector original:
--- Directorio de Empleados ---
ID      : 105
Nombre  : Ana
Salario : $3500.5

ID      : 102
Nombre  : Luis
Salario : $4200

ID      : 109
Nombre  : Carlos
Salario : $3900.75

ID      : 101
Nombre  : Maria
Salario : $4500.25

Vector ordenado:
--- Directorio de Empleados ---
ID      : 101
Nombre  : Maria
Salario : $4500.25

ID      : 102
Nombre  : Luis
Salario : $4200

ID      : 105
Nombre  : Ana
Salario : $3500.5

ID      : 109
Nombre  : Carlos
Salario : $3900.75

```

Ejercicio 5:

1. Crea un vector anidado que represente una cuadrícula de 16x16. Inicializa todos los valores en 0 (que significa disponible).
2. Escribe código para marcar los asientos (1,2), (2,4) y (3,8) con el valor 1 (ocupado). Recordar que los índices inician en 0.
3. Escribe una función o bloque de código con bucles anidados que imprima la matriz completa en forma de cuadrícula.
4. Calcula e imprime el total de asientos disponibles iterando por todas las filas y columnas del vector de vectores.

```
#include <iostream>
#include <vector>

using namespace std;

void imprimeMatriz(vector<vector<int>>& matriz);
int calculaDisponibles(vector<vector<int> >& matriz);

int main() {
    // 1. Crea un vector anidado que represente una cuadrícula de
    // 16x16. Inicializa todos los valores
    // en 0 (que significa disponible)
    vector<vector<int> > matriz(16, vector<int>(16, 0));

    // 2. Escribe código para marcar los asientos (1,2), (2,4) y (3,8)
    // con el valor 1 (ocupado).
    // Recordar que los índices inician en 0
    matriz[1][2] = 1;
    matriz[2][4] = 1;
    matriz[3][8] = 1;

    // 3. Escribe una función o bloque de código con bucles anidados
    // que imprima la matriz completa
    // en forma de cuadrícula.
    imprimeMatriz(matriz);

    // 4. Calcular e imprimir el total de asientos disponibles (valor
    // 0)
    int disponibles = calculaDisponibles(matriz);
    cout << "Total de asientos disponibles: " << disponibles << endl;
    cout << "Total de asientos ocupados: " << (16 * 16) - disponibles
    << endl;
```

```

        return 0;
    }

void imprimeMatriz(vector<vector<int> >& matriz) {
    cout << "--- Estado de la matriz (0: Libre, 1: Ocupado) ---" <<
endl;
    for (int i = 0; i < matriz.size(); i++) {
        for (int j = 0; j < matriz[i].size(); j++) {
            cout << matriz[i][j] << " ";
        }
        cout << endl;
    }
}

int calculaDisponibles(vector<vector<int> >& matriz) {
    int disponibles = 0;
    for (int i = 0; i < matriz.size(); i++) {
        for (int j = 0; j < matriz[i].size(); j++) {
            if (matriz[i][j] == 0) {
                disponibles++;
            }
        }
    }
    return disponibles;
}

```

Compilación y ejecución:

```

dalemanb@generic:/mnt/c/Users/dalem/Desktop/Cpp/C_plus_plus/Homework/Tarea2$ g++ Ejercicio5.cpp
dalemanb@generic:/mnt/c/Users/dalem/Desktop/Cpp/C_plus_plus/Homework/Tarea2$ ./a.out
--- Estado de la matriz (0: Libre, 1: Ocupado) ---
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Total de asientos disponibles: 253
Total de asientos ocupados: 3

```